

# МИНИСТЕРСТВО НАУКИ И ВЫСШЕГО ОБРАЗОВАНИЯ РОССИЙСКОЙ ФЕДЕРАЦИИ

Федеральное государственное бюджетное образовательное учреждение высшего образования  
«МИРЭА — Российский технологический университет»  
Институт кибербезопасности и цифровых технологий  
Кафедра информационной безопасности

## КУРСОВОЙ ПРОЕКТ

по дисциплине  
«Кибериммунная методология разработки ПО»

Тема: «Система обмена сообщениями с криптографической защитой  
информации на базе Kafka»

Выполнили: Ховроненко, Зюков, Кэп, группа 0Х-24  
Проверил: Чесалин Александр Николаевич

Москва, 2026

# Содержание

- 1 Введение
- 2 Постановка задачи
- 3 Общая характеристика разработанного решения
- 4 Архитектура и взаимодействие компонентов
- 5 Криптографическая защита и криптоконтейнеры
- 6 Kafka и модель топиков
- 7 База данных PostgreSQL
- 8 Мониторинг Grafana, Prometheus и Loki
- 9 Кибериммунный подход и политики безопасности
- 10 Паттерны проектирования
- 11 UML и draw.io диаграммы
- 12 Тестирование проектного кода
- 13 Инструкция по запуску

## Заклучение

# Приложение А. Исходный код

## 1 Введение

В рамках курсового проекта разработан учебный прототип LiteBroker — система обмена сообщениями с криптографической защитой информации. В отличие от обычного демонстрационного чата, проект показывает не только отправку текста между пользователями, но и архитектурный подход к построению системы с изолированными компонентами, брокером сообщений, локальной базой данных и наблюдаемостью.

Основная идея решения состоит в том, что пользовательский текст и телеметрия датчиков не должны попадать в инфраструктуру в открытом виде. Перед сохранением и публикацией данные преобразуются в криптоконтейнер AES-256-GCM. Такой контейнер содержит шифртекст, вектор инициализации, тег аутентификации и метаданные, необходимые для проверки целостности.

## 2 Постановка задачи

Перед проектом были поставлены следующие задачи:

- реализовать веб-интерфейс лёгкого мессенджера с выбором собеседника;
- организовать обмен сообщениями через Apache Kafka;
- сделать отдельный Kafka-топик для каждого чата;
- добавить отдельный топик для случайных показаний датчиков;
- реализовать хранение пользователей, сессий, сообщений и замеров в PostgreSQL;
- добавить криптографическую защиту сообщений и сенсорной телеметрии;
- подготовить Grafana-дашборды для датчиков и логов криптоконтейнеров;
- описать архитектуру политики, уровни доверия, UML-диаграммы и паттерны проектирования;
- подготовить проект к воспроизводимому запуску через Docker Compose.

## 3 Общая характеристика разработанного решения

LiteBroker построен как набор контейнеризованных сервисов. Пользователь работает через веб-интерфейс, серверное приложение принимает запросы, проверяет авторизацию, определяет Kafka-топик чата, создаёт криптоконтейнер и сохраняет его в PostgreSQL. Kafka используется как брокер событий, позволяющий показать разделение потоков: пользовательские сообщения и телеметрия датчиков передаются независимо.

## 4 Архитектура и взаимодействие компонентов

Архитектура проекта разделена на несколько контуров. Пользовательский контур считается недоверенным, так как пользовательский ввод может быть ошибочным или вредоносным. Серверный контур считается доверенной зоной политики: он принимает решения о допустимости операций, проверяет сессии и создаёт криптоконтейнеры. Kafka и PostgreSQL отнесены к ограниченно доверенной инфраструктуре: они необходимы для работы системы, но не должны получать открытый текст сообщений. Контур мониторинга является наблюдательным и не управляет логикой приложения.

Такое разделение соответствует принципу минимизации поверхности атаки: пользователь не обращается напрямую к Kafka или PostgreSQL, а все операции проходят через API. Даже если инфраструктурный компонент будет просматривать сохранённые записи, он увидит криптоконтейнер, а не исходный текст.

## 5 Криптографическая защита и криптоконтейнеры

Для защиты данных выбран алгоритм AES-256-GCM. AES обеспечивает симметричное шифрование, 256-битный ключ даёт высокий запас стойкости, а режим GCM дополнительно проверяет целостность и подлинность данных через authTag. Это важно для проекта, потому что недостаточно просто скрыть текст: необходимо обнаруживать изменение ciphertext, iv или metadata.

Важной особенностью реализации стала стабильная сериализация metadata при расчёте AAD. PostgreSQL хранит контейнеры как JSONB и может менять порядок ключей. Поэтому в сервере добавлена функция упорядочивания metadata перед шифрованием и расшифровкой. Это устраняет ошибку, при которой старый контейнер не проходил проверку AES-GCM из-за различий в порядке полей.

## 6 Kafka и модель топиков

Kafka используется для демонстрации брокерной архитектуры. Для каждого диалога формируется отдельный топик вида `litebroker.chat.alice__bob`. Имена пользователей сортируются, поэтому для пары `alice/bob` получается один общий топик независимо от направления отправки. Генератор датчиков пишет в отдельный топик `litebroker.sensors.random`.

Разделение топиков полезно для безопасности и сопровождаемости. Поток сообщений не смешивается с телеметрией, а Grafana может строить разные представления для разных типов данных.

## 7 База данных PostgreSQL

В актуальной версии проекта используется PostgreSQL. База данных хранит пользователей, сессии, сообщения и сенсорные замеры. Пароли не сохраняются в открытом виде: для них используется PBKDF2-SHA256 с солью. Сообщения хранятся в поле `crypto_container` типа JSONB, а сенсорные замеры — в `encrypted_payload`.

## 8 Мониторинг Grafana, Prometheus и Loki

Prometheus забирает метрики из endpoint `/metrics`. Loki и Promtail используются для доставки логов криптоконтейнеров в Grafana. В Grafana предусмотрены два смысловых представления: первое показывает динамику показаний датчиков, второе демонстрирует логи контейнеров сообщений. Такой подход помогает на защите проекта визуально показать, что система не просто отправляет текст, а работает с защищёнными контейнерами и наблюдаемой инфраструктурой.

## 9 Кибериммунный подход и политики безопасности

Кибериммунный подход в проекте выражен через ограничение доверия к компонентам и явную политику взаимодействия. Пользовательский ввод не считается доверенным, инфраструктура не получает открытый текст, а серверная логика выполняет роль контролирующего посредника.

## 10 Паттерны проектирования

## 11 UML и draw.io диаграммы

Для проекта подготовлены редактируемые draw.io диаграммы и растровые версии для вставки в отчёт. Диаграммы показывают классовую структуру, последовательность отправки сообщения, архитектуру контейнеров и распределение доверия между компонентами.

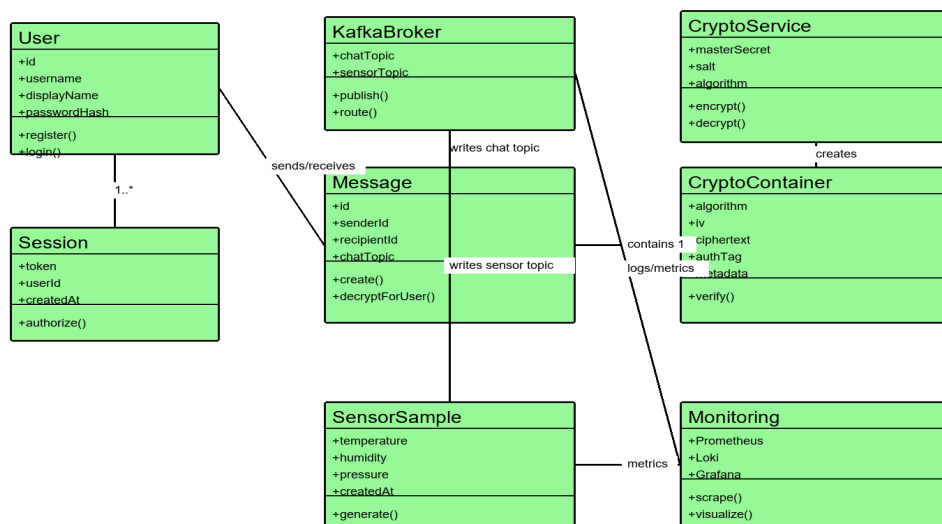


Рисунок 1 — UML-диаграмма классов LiteBroker

Sequence diagram: отправка защищённого сообщения

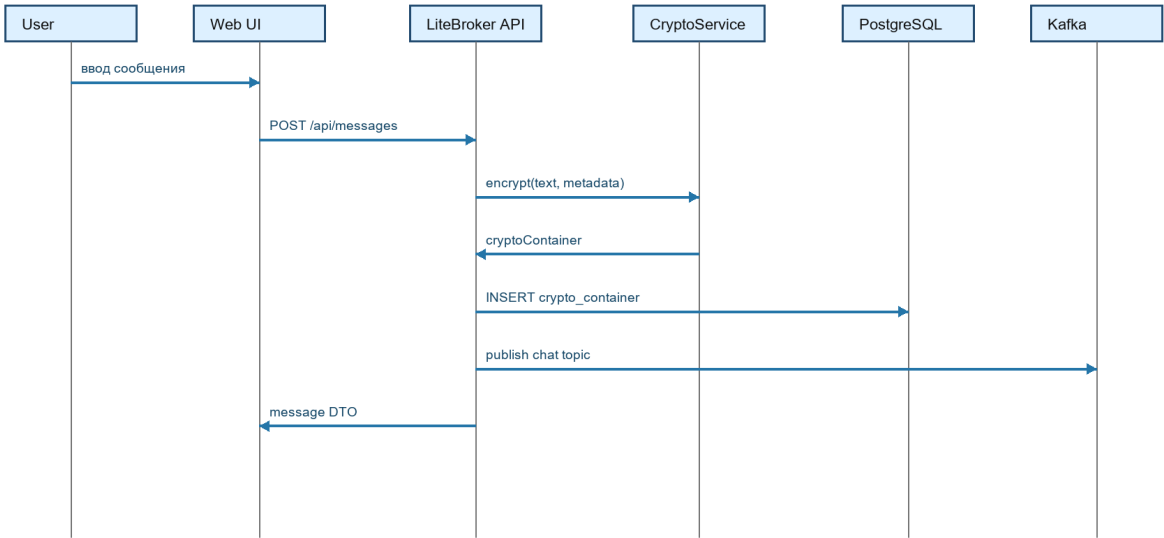


Рисунок 2 — Диаграмма последовательности отправки сообщения

Архитектура LiteBroker

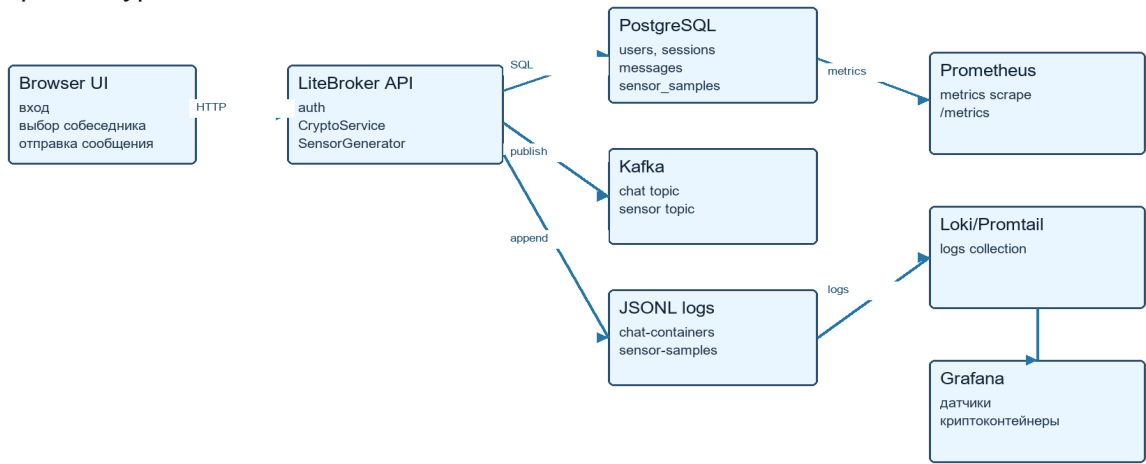


Рисунок 3 — Архитектура контейнерного решения

Таблица доменов безопасности LiteBroker

| Домен безопасности         | Уровень доверия        | Обоснование                                                                                 | Комментарий                           |
|----------------------------|------------------------|---------------------------------------------------------------------------------------------|---------------------------------------|
| 1. Browser UI              | недоверенный           | принимает пользовательский ввод, может отправить некорректные или вредоносные запросы       | все запросы проходят через API        |
| 2. LiteBroker API          | доверенный             | единственная точка принятия решений: авторизация, валидация, маршрутизация                  | повышает целостность данных           |
| 3. Auth/Session            | доверенный             | проверяет токены и не отдает разового_hash клиенту                                          | обязателен для всех бизнес-эндпоинт   |
| 4. CryptoService           | доверенный             | создает AES-256-GCM криптоконтейнеры и проверяет authTag                                    | критичный компонент защиты            |
| 5. PostgreSQL              | ограниченно доверенный | хранит данные, может быть прочитано администратором, поэтому payload хранится зашифрованным | не хранит открытый текст сообщений    |
| 6. Kafka Broker            | ограниченно доверенный | транспортный канал может быть просмотрен, но сообщения передаются как контейнеры            | разделяет chat topic и sensor topic   |
| 7. SensorGenerator         | доверенный             | генерирует учебные замеры без пользовательского ввода                                       | замеры также шифруются в БД           |
| 8. Prometheus/Loki/Grafana | недоверенный read-only | модуль наблюдаемости не должен менять бизнес-данные                                         | используется для контроля и защиты    |
| 9. Docker Network          | ограниченно доверенный | изолирует контейнеры, но не заменяет криптографическую защиту                               | задает границы взаимодействия         |
| 10. Operator/Admin         | доверенный             | имеет доступ к запуску стека и переменным окружения                                         | для промышленной версии нужны secrets |

Архитектура доменов доверия LiteBroker

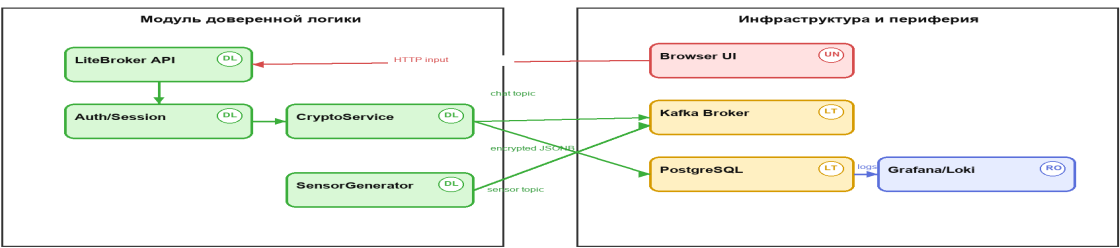


Рисунок 4 — Домены безопасности и уровни доверия

Домены доверия и контролируемые связи LiteBroker

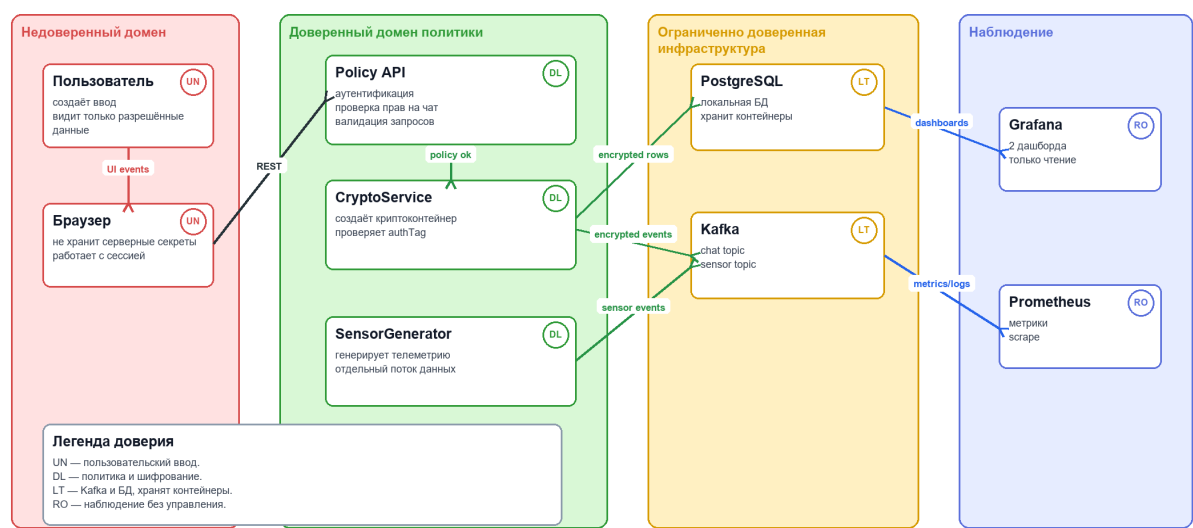


Рисунок 5 — Контролируемые связи и доверенность компонентов LiteBroker

12 Тестирование проектного кода

Тестирование проекта выполнялось как набор функциональных и инфраструктурных проверок:

- проверка сборки Docker-образа LiteBroker;
- запуск docker compose и проверка статуса контейнеров;
- проверка endpoint /api/health;
- проверка регистрации и входа по username/password;
- проверка выбора собеседника и отправки сообщения;
- проверка, что Enter отправляет сообщение, а Shift+Enter переносит строку;
- проверка, что сообщение возвращается в интерфейс как расшифрованный текст;
- проверка наличия crypto\_container с iv, ciphertext, authTag и metadata;
- проверка генерации сенсорных данных и метрик Prometheus;
- проверка отображения данных в Kafka UI и Grafana.

Отдельно проверялся негативный сценарий: если контейнер был зашифрован с другим ключом или изменённой metadata, AES-GCM не должен возвращать открытый текст. Это поведение считается корректным, так как режим GCM обязан обнаруживать нарушение целостности.

### 13 Инструкция по запуску

Открыть PowerShell в директории проекта: C:\Users\HONOR\Documents\New project 8.

Выполнить команду: `docker compose up --build -d`.

Открыть веб-интерфейс: <http://localhost:18091>.

Войти под тестовыми пользователями alice / 123456 или bob / 123456.

Для остановки выполнить: `docker compose down`.

Дополнительные сервисы: Kafka UI — <http://localhost:18089>, Grafana — <http://localhost:3001>, Prometheus — <http://localhost:9090>.



## Заключение

В результате выполненной работы был создан прототип защищённой системы обмена сообщениями LiteBroker. Проект объединяет веб-интерфейс, backend API, PostgreSQL, Kafka, генератор датчиков и стек мониторинга. Главная особенность проекта — хранение и передача сообщений в виде криптоконтейнеров, что напрямую соответствует теме криптографической защиты информации.

Работа демонстрирует применение кибериммунного подхода на уровне учебного прототипа: компоненты имеют разные уровни доверия, взаимодействуют по заданной политике, а открытый текст не передаётся в брокер и не сохраняется в базе данных. Подготовленные UML и draw.io диаграммы позволяют объяснить архитектуру проекта на защите и связать программную реализацию с выбранными политиками безопасности.

## Приложение А. Исходный код

В приложении приведены основные файлы проекта без README-файлов.

### docker-compose.yml

```
services:
  postgres:
    image: postgres:16-alpine
    container_name: litebroker-postgres
    ports:
      - "5432:5432"
    environment:
      POSTGRES_DB: litebroker
      POSTGRES_USER: litebroker
      POSTGRES_PASSWORD: litebroker
    volumes:
      - postgres-data:/var/lib/postgresql/data
    healthcheck:
      test: ["CMD-SHELL", "pg_isready -U litebroker -d litebroker"]
      interval: 5s
      timeout: 5s
      retries: 12

  kafka:
    image: apache/kafka:3.8.0
    container_name: litebroker-kafka
    ports:
      - "9092:9092"
    environment:
      KAFKA_NODE_ID: "1"
      KAFKA_PROCESS_ROLES: "broker,controller"
      KAFKA_CONTROLLER_QUORUM_VOTERS: "1@kafka:9093"
      KAFKA_LISTENERS: "PLAINTEXT://:9092,CONTROLLER://:9093"
      KAFKA_ADVERTISED_LISTENERS: "PLAINTEXT://kafka:9092"
      KAFKA_LISTENER_SECURITY_PROTOCOL_MAP:
        "CONTROLLER:PLAINTEXT,PLAINTEXT:PLAINTEXT"
      KAFKA_CONTROLLER_LISTENER_NAMES: "CONTROLLER"
      KAFKA_INTER_BROKER_LISTENER_NAME: "PLAINTEXT"
      KAFKA_OFFSETS_TOPIC_REPLICATION_FACTOR: "1"
      KAFKA_TRANSACTION_STATE_LOG_REPLICATION_FACTOR: "1"
      KAFKA_TRANSACTION_STATE_LOG_MIN_ISR: "1"
      KAFKA_GROUP_INITIAL_REBALANCE_DELAY_MS: "0"
      KAFKA_AUTO_CREATE_TOPICS_ENABLE: "true"
    volumes:
      - kafka-data:/var/lib/kafka/data

  kafka-ui:
    image: provectuslabs/kafka-ui:v0.7.2
    container_name: litebroker-kafka-ui
    depends_on:
      - kafka
```

ports:

- "18089:8080"

environment:

KAFKA\_CLUSTERS\_0\_NAME: litebroker

KAFKA\_CLUSTERS\_0\_BOOTSTRAPSERVERS: kafka:9092

litebroker:

build:

context: ./LiteBroker

container\_name: litebroker-app

depends\_on:

kafka:

condition: service\_started

postgres:

condition: service\_healthy

ports:

- "18091:18090"

environment:

LIGHT\_CRYPTO\_CHAT\_PORT: "18090"

LIGHT\_CRYPTO\_SECRET: litebroker-dev-secret-change-me

DATABASE\_URL: postgres://litebroker:litebroker@postgres:5432/litebroker

KAFKA\_BROKERS: kafka:9092

CHAT\_TOPIC\_PREFIX: litebroker.chat

SENSOR\_TOPIC: litebroker.sensors.random

SENSOR\_INTERVAL\_MS: "5000"

volumes:

- ./LiteBroker/logs:/app/logs

prometheus:

image: prom/prometheus:v2.55.1

container\_name: litebroker-prometheus

depends\_on:

- litebroker

ports:

- "9090:9090"

volumes:

- ./monitoring/prometheus.docker.yml:/etc/prometheus/prometheus.yml:ro

loki:

image: grafana/loki:3.2.1

container\_name: litebroker-loki

ports:

- "3100:3100"

command: ["-config.file=/etc/loki/local-config.yaml"]

promtail:

image: grafana/promtail:3.2.1

container\_name: litebroker-promtail

depends\_on:

- loki

```
- litebroker
volumes:
- ./LiteBroker/logs:/var/log/litebroker:ro
- ./monitoring/promtail.yml:/etc/promtail/config.yml:ro
command: ["-config.file=/etc/promtail/config.yml"]
```

```
grafana:
image: grafana/grafana:11.3.1
container_name: litebroker-grafana
depends_on:
- prometheus
- loki
ports:
- "3001:3000"
environment:
GF_SECURITY_ADMIN_USER: admin
GF_SECURITY_ADMIN_PASSWORD: admin
volumes:
- grafana-data:/var/lib/grafana
- ./monitoring/grafana/provisioning:/etc/grafana/provisioning:ro
- ./monitoring/grafana/dashboards:/var/lib/grafana/dashboards:ro
```

```
volumes:
postgres-data:
kafka-data:
grafana-data:
```

```
LiteBroker\server.mjs
```

```
import { createServer } from "node:http";
import { appendFile, mkdir, readFile } from "node:fs/promises";
import { existsSync } from "node:fs";
import { createCipheriv, createDecipheriv, pbkdf2Sync, randomBytes, timingSafeEqual } from "node:crypto";
import { dirname, extname, join } from "node:path";
import { fileURLToPath } from "node:url";
import { Kafka } from "kafkajs";
import pg from "pg";

const { Pool } = pg;

const __dirname = dirname(fileURLToPath(import.meta.url));
const PORT = Number(process.env.LIGHT_CRYPT_CHAT_PORT || 18090);
const LOG_DIR = join(__dirname, "logs");
const PUBLIC_DIR = join(__dirname, "public");
const MASTER_SECRET = process.env.LIGHT_CRYPT_SECRET || "dev-secret-change-me";
const KAFKA_BROKERS = (process.env.KAFKA_BROKERS || "localhost:9092").split(",");
const SENSOR_TOPIC = process.env.SENSOR_TOPIC || "litebroker.sensors.random";
const CHAT_TOPIC_PREFIX = process.env.CHAT_TOPIC_PREFIX || "litebroker.chat";
const DATABASE_URL = process.env.DATABASE_URL || "postgres://litebroker:litebroker@localhost:5432/litebroker";

let pool;
let producer;
let kafkaReady = false;
let dbReady = false;
let sensorTimer;

function id(prefix) {
return `${prefix}_${Date.now().toString(36)}_${randomBytes(4).toString("hex")}`;
}

function now() {
return new Date().toISOString();
}

function normalizeUsername(value = "") {
return String(value).trim().replace(/^@+/, "").toLowerCase();
}
```

```

}

function safeTopicPart(value) {
return normalizeUsername(value).replace(/[^a-z0-9_]/g, "_") || "unknown";
}

function chatTopic(firstUsername, secondUsername) {
return `${CHAT_TOPIC_PREFIX}.${[safeTopicPart(firstUsername), safeTopicPart(secondUsername)].sort().join("__")}`;
}

function passwordHash(password, salt = randomBytes(16).toString("hex")) {
return {
salt,
hash: pbkdf2Sync(String(password), salt, 120_000, 32, "sha256").toString("hex"),
};
}

function passwordOk(password, user) {
const next = passwordHash(password, user.password_salt).hash;
return timingSafeEqual(Buffer.from(next, "hex"), Buffer.from(user.password_hash, "hex"));
}

function cryptoKey() {
return pbkdf2Sync(MASTER_SECRET, "light-crypto-chat-v2-kafka", 120_000, 32, "sha256");
}

function orderedMetadata(metadata = {}) {
if (metadata.messageId) {
return {
messageId: metadata.messageId,
senderId: metadata.senderId,
recipientId: metadata.recipientId,
chatTopic: metadata.chatTopic,
createdAt: metadata.createdAt,
};
}

if (metadata.sampleId) {
return {
sampleId: metadata.sampleId,
topic: metadata.topic,
createdAt: metadata.createdAt,
storage: metadata.storage,
};
}

return Object.keys(metadata)
.sort()
.reduce((result, key) => {
result[key] = metadata[key];
return result;
}, {});
}

function aadBuffer(metadata) {
return Buffer.from(JSON.stringify(orderedMetadata(metadata)), "utf8");
}

function encryptContainer(value, metadata) {
const iv = randomBytes(12);
const cipher = createCipheriv("aes-256-gcm", cryptoKey(), iv);
cipher.setAAD(aadBuffer(metadata));
const ciphertext = Buffer.concat([cipher.update(String(value), "utf8"), cipher.final()]);
return {
version: 2,
algorithm: "AES-256-GCM",
iv: iv.toString("base64"),
ciphertext: ciphertext.toString("base64"),
authTag: cipher.getAuthTag().toString("base64"),
metadata,
};
}

```

```

function decryptContainer(container) {
const decipher = createDecipheriv("aes-256-gcm", cryptoKey(), Buffer.from(container.iv, "base64"));
decipher.setAAD(aadBuffer(container.metadata));
decipher.setAuthTag(Buffer.from(container.authTag, "base64"));
return Buffer.concat([
decipher.update(Buffer.from(container.ciphertext, "base64")),
decipher.final(),
]).toString("utf8");
}

```

```

function encryptMessage(plainText, metadata) {
return encryptContainer(plainText, metadata);
}

```

```

function decryptMessage(container) {
return decryptContainer(container);
}

```

```

function encryptJsonPayload(payload, metadata) {
return encryptContainer(JSON.stringify(payload), metadata);
}

```

```

function decryptJsonPayload(container) {
return JSON.parse(decryptContainer(container));
}

```

```

async function query(sql, params = []) {
const database = await openDb();
return database.query(sql, params);
}

```

```

async function openDb() {
if (pool) return pool;
await mkdir(LOG_DIR, { recursive: true });
pool = new Pool({
connectionString: DATABASE_URL,
max: 10,
idleTimeoutMillis: 30_000,
});
pool.on("error", (error) => {
dbReady = false;
console.error("PostgreSQL pool error:", error.message);
});
}

```

```

await pool.query(`
CREATE TABLE IF NOT EXISTS users (
id TEXT PRIMARY KEY,
username TEXT NOT NULL UNIQUE,
display_name TEXT NOT NULL,
password_hash TEXT NOT NULL,

```

```
password_salt TEXT NOT NULL,  
created_at TIMESTAMPTZ NOT NULL  
);
```

```
CREATE TABLE IF NOT EXISTS sessions (  
token TEXT PRIMARY KEY,  
user_id TEXT NOT NULL REFERENCES users(id) ON DELETE CASCADE,  
created_at TIMESTAMPTZ NOT NULL  
);
```

```
CREATE TABLE IF NOT EXISTS messages (  
id TEXT PRIMARY KEY,  
chat_topic TEXT NOT NULL,  
sender_id TEXT NOT NULL REFERENCES users(id) ON DELETE CASCADE,  
sender_name TEXT NOT NULL,  
recipient_id TEXT NOT NULL REFERENCES users(id) ON DELETE CASCADE,  
recipient_name TEXT NOT NULL,  
crypto_container JSONB NOT NULL,  
created_at TIMESTAMPTZ NOT NULL,  
kafka_offset TEXT  
);
```

```
CREATE INDEX IF NOT EXISTS idx_messages_users_created  
ON messages (sender_id, recipient_id, created_at);
```

```
CREATE INDEX IF NOT EXISTS idx_messages_topic  
ON messages (chat_topic);
```

```
CREATE TABLE IF NOT EXISTS sensor_samples (  
id TEXT PRIMARY KEY,  
topic TEXT NOT NULL,  
temperature DOUBLE PRECISION,  
humidity DOUBLE PRECISION,  
pressure DOUBLE PRECISION,  
encrypted_payload JSONB,  
created_at TIMESTAMPTZ NOT NULL  
);
```

```
CREATE INDEX IF NOT EXISTS idx_sensor_samples_created  
ON sensor_samples (created_at DESC);  
`);  
await migrateSensorEncryption();  
dbReady = true;  
await seedUsers();  
return pool;  
}
```

```
async function migrateSensorEncryption() {  
await pool.query(`  
ALTER TABLE sensor_samples ADD COLUMN IF NOT EXISTS encrypted_payload JSONB;
```

```

ALTER TABLE sensor_samples ALTER COLUMN temperature DROP NOT NULL;
ALTER TABLE sensor_samples ALTER COLUMN humidity DROP NOT NULL;
ALTER TABLE sensor_samples ALTER COLUMN pressure DROP NOT NULL;
`);

```

```

const { rows } = await pool.query(`
SELECT id, topic, temperature, humidity, pressure, created_at
FROM sensor_samples
WHERE encrypted_payload IS NULL
AND temperature IS NOT NULL
AND humidity IS NOT NULL
AND pressure IS NOT NULL
ORDER BY created_at ASC
LIMIT 2000
`);

```

```

for (const row of rows) {
const createdAt = new Date(row.created_at).toISOString();
const encryptedPayload = encryptJsonPayload(
{
temperature: Number(row.temperature),
humidity: Number(row.humidity),
pressure: Number(row.pressure),
},
{
sampleId: row.id,
topic: row.topic,
createdAt,
storage: "postgres.sensor_samples.encrypted_payload",
},
);
await pool.query(
`UPDATE sensor_samples
SET encrypted_payload = $1::jsonb,
temperature = NULL,
humidity = NULL,
pressure = NULL
WHERE id = $2`,
[JSON.stringify(encryptedPayload), row.id],
);
}
}

```

```

async function seedUsers() {
const { rows } = await pool.query("SELECT COUNT(*)::int AS total FROM users");
if (rows[0].total > 0) return;

```

```

for (const [username, displayName] of [
["alice", "Алиса"],
["bob", "Боб"],

```



```

["sensor_admin", "Администратор датчиков"],
]) {
const hashed = passwordHash("123456");
await pool.query(
`INSERT INTO users (id, username, display_name, password_hash, password_salt, created_at)
VALUES ($1, $2, $3, $4, $5, $6)`,
[id("user"), username, displayName, hashed.hash, hashed.salt, now()],
);
}
}

```

```

async function kafkaProducer() {
if (producer) return producer;
const kafka = new Kafka({ clientId: "litebroker-api", brokers: KAFKA_BROKERS });
producer = kafka.producer();
try {
await producer.connect();
kafkaReady = true;
} catch (error) {
kafkaReady = false;
console.error("Kafka unavailable, local persistence still works:", error.message);
}
return producer;
}

```

```

async function publishKafka(topic, key, value) {
const currentProducer = await kafkaProducer();
if (!kafkaReady) {
try {
await currentProducer.connect();
kafkaReady = true;
} catch (error) {
return { ok: false, reason: `kafka unavailable: ${error.message}` };
}
}
try {
const result = await currentProducer.send({ topic, messages: [{ key, value: JSON.stringify(value) }] });
return { ok: true, offset: result?.[0]?.baseOffset || "" };
} catch (error) {
kafkaReady = false;
console.error("Kafka publish failed:", error.message);
return { ok: false, reason: error.message };
}
}

```

```

async function body(request) {
const chunks = [];
for await (const chunk of request) chunks.push(chunk);
if (!chunks.length) return {};
try {

```

```

return JSON.parse(Buffer.concat(chunks).toString("utf8"));
} catch {
const badRequest = new Error("Invalid JSON");
badRequest.statusCode = 400;
throw badRequest;
}
}

```

```

function json(response, status, payload) {
response.writeHead(status, {
"content-type": "application/json; charset=utf-8",
"access-control-allow-origin": "*",
"access-control-allow-methods": "GET,POST,OPTIONS",
"access-control-allow-headers": "content-type, authorization",
});
response.end(JSON.stringify(payload));
}

```

```

function text(response, status, payload) {
response.writeHead(status, {
"content-type": "text/plain; charset=utf-8",
"access-control-allow-origin": "*",
});
response.end(payload);
}

```

```

function error(response, status, message) {
json(response, status, { error: message });
}

```

```

async function currentUser(request) {
const token = (request.headers.authorization || "").replace(/^Bearer /, "");
if (!token) return null;
const { rows: sessions } = await query("SELECT * FROM sessions WHERE token = $1", [token]);
if (!sessions.length) return null;
const { rows: users } = await query("SELECT * FROM users WHERE id = $1", [sessions[0].user_id]);
return users[0] || null;
}

```

```

function publicUser(user) {
return { id: user.id, username: `@${user.username}`, displayName: user.display_name };
}

```

```

function messageDto(row) {
const container = typeof row.crypto_container === "string" ? JSON.parse(row.crypto_container) :
row.crypto_container;
let textValue = "";
try {
textValue = decryptMessage(container);
}

```

```

    } catch {
    textValue = "[сообщение не удалось расшифровать]";
    }
    return {
    id: row.id,
    chatTopic: row.chat_topic,
    senderId: row.sender_id,
    senderName: row.sender_name,
    recipientId: row.recipient_id,
    recipientName: row.recipient_name,
    createdAt: new Date(row.created_at).toISOString(),
    text: textValue,
    cryptoContainer: container,
    kafkaOffset: row.kafka_offset || "",
    };
}

```

```

function sensorDto(row) {
const createdAt = new Date(row.created_at).toISOString();
const container = typeof row.encrypted_payload === "string" ? JSON.parse(row.encrypted_payload) :
row.encrypted_payload;

```

```

if (container) {
try {
const payload = decryptJsonPayload(container);
return {
id: row.id,
topic: row.topic,
temperature: Number(payload.temperature),
humidity: Number(payload.humidity),
pressure: Number(payload.pressure),
created_at: createdAt,
encrypted: true,
};
} catch {
return {
id: row.id,
topic: row.topic,
temperature: 0,
humidity: 0,
pressure: 0,
created_at: createdAt,
encrypted: true,
decryptError: true,
};
}
}

```

```

return {
id: row.id,

```

```

topic: row.topic,
temperature: Number(row.temperature),
humidity: Number(row.humidity),
pressure: Number(row.pressure),
created_at: createdAt,
encrypted: false,
};
}

```

```

async function userMessages(userId, peerId = "") {
const params = peerId ? [userId, userId, peerId, peerId] : [userId, userId];
const sql = peerId
? `SELECT * FROM messages
WHERE (sender_id = $1 OR recipient_id = $2) AND (sender_id = $3 OR recipient_id = $4)
ORDER BY created_at ASC LIMIT 200`
: `SELECT * FROM messages
WHERE sender_id = $1 OR recipient_id = $2
ORDER BY created_at ASC LIMIT 200`;
const { rows } = await query(sql, params);
return rows.map(messageDto);
}

```

```

async function appendChatLog(event) {
await appendFile(join(LOG_DIR, "chat-containers.jsonl"), `${JSON.stringify(event)}\n`, "utf8");
}

```

```

async function appendSensorLog(event) {
await appendFile(join(LOG_DIR, "sensor-samples.jsonl"), `${JSON.stringify(event)}\n`, "utf8");
}

```

```

async function createSensorSample() {
const createdAt = now();
const sample = {
id: id("sensor"),
topic: SENSOR_TOPIC,
temperature: Number((20 + Math.random() * 12).toFixed(2)),
humidity: Number((35 + Math.random() * 45).toFixed(2)),
pressure: Number((735 + Math.random() * 35).toFixed(2)),
createdAt,
};
const encryptedPayload = encryptJsonPayload(
{
temperature: sample.temperature,
humidity: sample.humidity,
pressure: sample.pressure,
},
{
sampleId: sample.id,
topic: sample.topic,
}

```

```

    createdAt,
    storage: "postgres.sensor_samples.encrypted_payload",
  },
);
await query(
  `INSERT INTO sensor_samples (id, topic, encrypted_payload, created_at)
  VALUES ($1, $2, $3::jsonb, $4)`,
  [sample.id, sample.topic, JSON.stringify(encryptedPayload), sample.createdAt],
);
await publishKafka(SENSOR_TOPIC, "random-sensor", { eventType: "sensor.sample", ...sample });
await appendSensorLog({ eventType: "sensor.sample", ...sample });
}

function startSensorGenerator() {
  if (sensorTimer) return;
  sensorTimer = setInterval(() => {
    createSensorSample().catch((generationError) => console.error("sensor generator failed:", generationError));
  }, Number(process.env.SENSOR_INTERVAL_MS || 5000));
}

async function metrics() {
  const [{ rows: userRows }, { rows: messageRows }, { rows: sensorRows }, { rows: lastSensorRows }, { rows:
  topicRows }] = await Promise.all([
    query("SELECT COUNT(*)::int AS total FROM users"),
    query("SELECT COUNT(*)::int AS total FROM messages"),
    query("SELECT COUNT(*)::int AS total FROM sensor_samples"),
    query("SELECT * FROM sensor_samples ORDER BY created_at DESC LIMIT 1"),
    query("SELECT chat_topic, COUNT(*)::int AS total FROM messages GROUP BY chat_topic"),
  ]);
  const lines = [
    "# HELP litebroker_users_total Total users",
    "# TYPE litebroker_users_total gauge",
    `litebroker_users_total ${userRows[0].total}`,
    "# HELP litebroker_messages_total Total chat messages",
    "# TYPE litebroker_messages_total gauge",
    `litebroker_messages_total ${messageRows[0].total}`,
    "# HELP litebroker_sensor_samples_total Total sensor samples",
    "# TYPE litebroker_sensor_samples_total gauge",
    `litebroker_sensor_samples_total ${sensorRows[0].total}`,
    "# HELP litebroker_kafka_ready Kafka producer status",
    "# TYPE litebroker_kafka_ready gauge",
    `litebroker_kafka_ready ${kafkaReady ? 1 : 0}`,
    "# HELP litebroker_postgres_ready PostgreSQL database status",
    "# TYPE litebroker_postgres_ready gauge",
    `litebroker_postgres_ready ${dbReady ? 1 : 0}`,
  ];
  for (const row of topicRows) {
    lines.push(`litebroker_chat_messages_total{topic="${row.chat_topic}"} ${row.total}`);
  }
  const lastSensor = lastSensorRows[0] ? sensorDto(lastSensorRows[0]) : null;

```

```

if (lastSensor) {
  lines.push(`litebroker_sensor_temperature_celsius ${lastSensor.temperature}`);
  lines.push(`litebroker_sensor_humidity_percent ${lastSensor.humidity}`);
  lines.push(`litebroker_sensor_pressure_mmhg ${lastSensor.pressure}`);
}
return `${lines.join("\n")}\n`;
}

async function api(request, response, url) {
  await openDb();

  if (url.pathname === "/api/health") {
    return json(response, 200, {
      ok: true,
      port: PORT,
      database: "postgresql",
      postgres: dbReady,
      kafka: { ready: kafkaReady },
      encryption: "AES-256-GCM",
      databaseEncryption: {
        messages: "AES-256-GCM crypto_container JSONB",
        sensors: "AES-256-GCM encrypted_payload JSONB",
        passwords: "PBKDF2-SHA256 salted hash",
      },
    });
  }

  if (url.pathname === "/metrics") {
    return text(response, 200, await metrics());
  }

  if (url.pathname === "/api/auth/register" && request.method === "POST") {
    const input = await body(request);
    const username = normalizeUsername(input.username);
    if (!/^[a-z0-9_]{3,24}$/.test(username)) return error(response, 400, "Username: 3-24 символа, латиница, цифры и _");
    if (String(input.password || "").length < 6) return error(response, 400, "Пароль минимум 6 символов");
    const { rows: exists } = await query("SELECT id FROM users WHERE username = $1", [username]);
    if (exists.length) return error(response, 409, "Username уже занят");

    const hashed = passwordHash(input.password);
    const user = {
      id: id("user"),
      username,
      displayName: String(input.displayName || username).trim().slice(0, 40),
      createdAt: now(),
    };
    await query(
      `INSERT INTO users (id, username, display_name, password_hash, password_salt, created_at)
      VALUES ($1, $2, $3, $4, $5, $6)`
    );
  }
}

```

```

[user.id, user.username, user.displayName, hashed.hash, hashed.salt, user.createdAt],
);
const token = id("session");
await query("INSERT INTO sessions (token, user_id, created_at) VALUES ($1, $2, $3)", [token, user.id,
now()]);
return json(response, 201, { token, user: publicUser({ ...user, display_name: user.displayName }) });
}

if (url.pathname === "/api/auth/login" && request.method === "POST") {
const input = await body(request);
const { rows } = await query("SELECT * FROM users WHERE username = $1",
[normalizeUsername(input.username)]);
const user = rows[0];
if (!user || !passwordOk(input.password || "", user)) return error(response, 401, "Неверный username или
пароль");
const token = id("session");
await query("INSERT INTO sessions (token, user_id, created_at) VALUES ($1, $2, $3)", [token, user.id,
now()]);
return json(response, 200, { token, user: publicUser(user) });
}

const user = await currentUser(request);
if (!user) return error(response, 401, "Нужна авторизация");

if (url.pathname === "/api/users" && request.method === "GET") {
const { rows } = await query("SELECT * FROM users WHERE id != $1 ORDER BY username ASC",
[user.id]);
return json(response, 200, { user: publicUser(user), users: rows.map(publicUser) });
}

if (url.pathname === "/api/messages" && request.method === "GET") {
const peerId = url.searchParams.get("peerId") || "";
return json(response, 200, { user: publicUser(user), messages: await userMessages(user.id, peerId) });
}

if (url.pathname === "/api/messages" && request.method === "POST") {
const input = await body(request);
const { rows: recipients } = await query("SELECT * FROM users WHERE id = $1", [String(input.recipientId || "")]);
const recipient = recipients[0];
if (!recipient || recipient.id === user.id) return error(response, 400, "Выберите собеседника");
const messageText = String(input.text || "").trim();
if (!messageText) return error(response, 400, "Сообщение пустое");
const messageId = id("msg");
const createdAt = now();
const topic = chatTopic(user.username, recipient.username);
const metadata = { messageId, senderId: user.id, recipientId: recipient.id, chatTopic: topic, createdAt };
const container = encryptMessage(messageText, metadata);
const event = { eventType: "chat.message.container", id: messageId, chatTopic: topic, createdAt, cryptoContainer: container };
const kafkaResult = await publishKafka(topic, topic, event);

await query(
`INSERT INTO messages (
id, chat_topic, sender_id, sender_name, recipient_id, recipient_name,
crypto_container, created_at, kafka_offset
)
VALUES ($1, $2, $3, $4, $5, $6, $7::jsonb, $8, $9)`,
[
messageId,
topic,
user.id,
user.display_name,
recipient.id,

```

```

    recipient.display_name,
    JSON.stringify(container),
    createdAt,
    kafkaResult.offset || "",
  ],
);
await appendChatLog(event);
const { rows } = await query("SELECT * FROM messages WHERE id = $1", [messageId]);
return json(response, 201, { message: messageDto(rows[0]), kafka: kafkaResult });
}

if (url.pathname === "/api/sensors" && request.method === "GET") {
  const { rows } = await query("SELECT * FROM sensor_samples ORDER BY created_at DESC LIMIT 50");
  return json(response, 200, {
    topic: SENSOR_TOPIC,
    samples: rows.reverse().map(sensorDto),
  });
}

return error(response, 404, "Маршрут не найден");
}

async function staticFile(response, pathname) {
  const file = pathname === "/" ? "index.html" : pathname.slice(1);
  const target = join(PUBLIC_DIR, file);
  if (!target.startsWith(PUBLIC_DIR) || !existsSync(target)) return false;
  const types = { ".html": "text/html", ".css": "text/css", ".js": "text/javascript" };
  response.writeHead(200, { "content-type": `${types[extname(target)]} || text/plain`; charset=utf-8` });
  response.end(await readFile(target));
  return true;
}

await openDb();
await kafkaProducer();
startSensorGenerator();

createServer(async (request, response) => {
  try {
    if (request.method === "OPTIONS") return json(response, 204, {});
    const url = new URL(request.url || "/", `http://${request.headers.host}`);
    if (url.pathname.startsWith("/api/") || url.pathname === "/metrics") return api(request, response, url);
    if (await staticFile(response, url.pathname)) return;
    error(response, 404, "Файл не найден");
  } catch (exception) {
    console.error(
      r(exception);
      if (exception.statusCode === 400) return error(response, 400, "Некорректный JSON в запросе");
      error(response, 500, "Ошибка сервера");
    }
  }
}).listen(PORT, "0.0.0.0", () => {
  console.log(` LiteBroker Kafka chat listening on http://localhost:${PORT} `);
});

```

LiteBroker\public\index.html

```

<!doctype html>
<html lang="ru">
<head>
<meta charset="UTF-8" />
<meta name="viewport" content="width=device-width, initial-scale=1.0" />
<title>LiteBroker</title>
<link rel="stylesheet" href="/styles.css" />
</head>
<body>
<main class="shell">
<section id="auth" class="panel auth-panel">
<p class="eyebrow">Kafka + AES-256-GCM</p>
<h1>LiteBroker</h1>
<p class="lead">
  Легкая версия CryptoBroker: каждый диалог публикуется в отдельный Kafka-топик,
  а генератор датчиков пишет замеры в отдельный поток.
</p>
<div class="tabs" role="tablist" aria-label="Авторизация">
<button id="login-tab" class="active" type="button">Вход</button>
<button id="register-tab" type="button">Регистрация</button>
</div>
<form id="auth-form" class="stack">

```



```

<label id="display-name-wrap">
Имя
<input id="display-name" placeholder="Например, Андрей" autocomplete="name" />
</label>
<label>
Username
<input id="username" placeholder="andrey_01" autocomplete="username" />
</label>
<label>
Пароль
<input id="password" placeholder="минимум 6 символов" type="password" autocomplete="current-password" />
</label>
<button id="auth-submit" type="submit">Войти</button>
</form>
<p id="auth-error" class="error" role="alert"></p>
<p class="hint">Тестовые аккаунты: alice / 123456, bob / 123456.</p>
</section>

<section id="app" class="lab-layout" hidden>
<header class="lab-topbar">
<div class="select-wrap">
<button id="select-user-button" class="select-user-button" type="button">Выбор<br />пользователя</button>
<div id="user-popover" class="user-popover" hidden>
<label>
поиск
<input id="user-search" placeholder="username или имя" autocomplete="off" />
</label>
<div id="users" class="users"></div>
<div class="sensor-card">
<strong>sensor topic</strong>
<p id="sensor-topic"></p>
<div id="sensor-list" class="sensor-list"></div>
</div>
<button id="logout" class="logout-button" type="button">Выйти</button>
</div>
</div>

<div>
<div id="chat-title" class="peer-name">USERNAME</div>
<div id="session-label" class="session-label"></div>
</div>

<div class="crypto-method">
<span>метод шифрования</span>
<strong>AES-256-GCM</strong>
<small id="server-status">проверка...</small>
</div>

<div class="crypto-heading">crypto containers info</div>
</header>

<section class="chat-stage">
<main class="chat-canvas">
<div id="chat-topic" class="topic-line">Для каждого диалога создается отдельный Kafka-топик.</div>
<div id="empty-state" class="empty-state">
<h3>Выберите пользователя</h3>
<p>После выбора собес
едника сообщение будет сохранено как криптоконтейнер и отправлено в Kafka.</p>
</div>
<div id="messages" class="messages" hidden></div>
<form id="message-form" class="composer">
<textarea
id="message-text"
placeholder="Сообщение будет сохранено как криптоконтейнер..."
rows="2"
disabled
></textarea>
<button id="send-button" type="submit" disabled>отправить</button>
</form>
</main>

```

```

<aside id="crypto-view" class="crypto-feed">
<div id="container-view" class="container-feed"></div>
</aside>
</section>
</section>
</main>
<script src="/app.js"></script>
</body>
</html>

```

## LiteBroker\public\app.js

```

const els = {
  auth: document.querySelector("#auth"),
  app: document.querySelector("#app"),
  loginTab: document.querySelector("#login-tab"),
  registerTab: document.querySelector("#register-tab"),
  authForm: document.querySelector("#auth-form"),
  displayNameWrap: document.querySelector("#display-name-wrap"),
  displayName: document.querySelector("#display-name"),
  username: document.querySelector("#username"),
  password: document.querySelector("#password"),
  authSubmit: document.querySelector("#auth-submit"),
  authError: document.querySelector("#auth-error"),
  sessionLabel: document.querySelector("#session-label"),
  logout: document.querySelector("#logout"),
  selectUserButton: document.querySelector("#select-user-button"),
  userPopover: document.querySelector("#user-popover"),
  users: document.querySelector("#users"),
  userSearch: document.querySelector("#user-search"),
  chatTitle: document.querySelector("#chat-title"),
  chatTopic: document.querySelector("#chat-topic"),
  serverStatus: document.querySelector("#server-status"),
  emptyState: document.querySelector("#empty-state"),
  messages: document.querySelector("#messages"),
  messageForm: document.querySelector("#message-form"),
  messageText: document.querySelector("#message-text"),
  sendButton: document.querySelector("#send-button"),
  containerView: document.querySelector("#container-view"),
  sensorTopic: document.querySelector("#sensor-topic"),
  sensorList: document.querySelector("#sensor-list"),
};

let mode = "login";
let token = localStorage.getItem("litebroker.token") || "";
let me = null;
let users = [];
let selectedPeer = null;
let messages = [];

function setMode(nextMode) {
  mode = nextMode;
  els.loginTab.classList.toggle("active", mode === "login");
  els.registerTab.classList.toggle("active", mode === "register");
  els.displayNameWrap.hidden = mode === "login";
  els.authSubmit.textContent = mode === "login" ? "Войти" : "Создать аккаунт";
  els.authError.textContent = "";
}

async function request(path, options = {}) {
  const response = await fetch(path, {
    ...options,
    headers: {
      "content-type": "application/json",
      ...(token ? { authorization: `Bearer ${token}` } : {}),
      ...(options.headers || {}),
    },
  });
  const data = await response.json();
  if (!response.ok) throw new Error(data.error || "Ошибка запроса");
  return data;
}

function showApp(user) {
  me = user;

```

```

els.auth.hidden = true;
els.app.hidden = false;
els.sessionLabel.textContent = `${user.displayName} ${user.username}`;
}

function logout() {
token = "";
me = null;
selectedPeer = null;
localStorage.removeItem("litebroker.token");
els.app.hidden = true;
els.auth.hidden = false;
}

function renderUsers() {
const query = els.userSearch.value.trim().toLowerCase();
const filtered = users.filter(user => {
const haystack = `${user.displayName} ${user.username}`.toLowerCase();
return !query || haystack.includes(query);
});

els.users.innerHTML = "";
if (!filtered.length) {
els.users.innerHTML = `<div class="empty-mini">Пользователь не найден</div>`;
return;
}

for (const user of filtered) {
const button = document.
createElement("button");
button.type = "button";
button.className = `user-item ${selectedPeer?.id === user.id ? "active" : ""}`;
button.innerHTML = `
<span class="avatar">${initials(user.displayName)}</span>
<span>
<strong></strong>
<small></small>
</span>
`;
button.querySelector("strong").textContent = user.displayName;
button.querySelector("small").textContent = user.username;
button.addEventListener("click", () => {
selectPeer(user);
els.userPopover.hidden = true;
});
els.users.append(button);
}
}

function initials(name) {
return name
.split(/\s+/)
.filter(Boolean)
.slice(0, 2)
.map((part) => part[0].toUpperCase())
.join("");
}

async function selectPeer(user) {
selectedPeer = user;
els.chatTitle.textContent = user.username.replace(/^@/, "");
}

```

```

els.chatTopic.textContent = "загрузка топика...";
els.messageText.disabled = false;
els.sendButton.disabled = false;
renderUsers();
await loadMessages();
els.messageText.focus();
}

function renderMessages() {
els.messages.innerHTML = "";
els.emptyState.hidden = Boolean(selectedPeer);
els.messages.hidden = !selectedPeer;

if (selectedPeer && !messages.length) {
els.messages.innerHTML = `<div class="empty-state inline"><h3>Сообщений пока нет</h3><p>Отправьте
первое сообщение в защищенный Kafka-топик.</p></div>`;
els.containerView.innerHTML = "";
return;
}

for (const message of messages) {
const own = message.senderId === me.id;
const item = document.createElement("article");
item.className = `message ${own ? "own" : ""}`;
item.innerHTML = `
<p></p>
<footer><span class="msg-time"></span><span class="msg-method"></span></footer>
`;
item.querySelector("p").textContent = message.text;
item.querySelector(".msg-time").textContent = new Date(message.createdAt).toLocaleString([], {
day: "2-digit",
month: "2-digit",
hour: "2-digit",
minute: "2-digit",
});
item.querySelector(".msg-method").textContent = message.cryptoContainer.algorithm;
item.addEventListener("click", () => {
highlightContainer(message.id);
});
els.messages.append(item);
}

const last = messages.at(-1);
renderCryptoFeed(messages, last?.id);
els.messages.scrollTop = els.messages.scrollHeight;
}

function renderCryptoFeed(items, activeId = "") {
els.containerView.innerHTML = "";
for (const message of items.slice().reverse()) {

```

```

const card = document.createElement("article");
card.className = `container-card ${message.id === activeId ? "active" : ""}`;
card.dataset.messageId = message.id;
card.innerHTML = `
<header>
<strong></strong>
<span></span>
</header>
<pre></pre>
`;
card.querySelector("strong").textContent = message.text || "[пусто]";
card.querySelector("span").textContent = new Date(message.createdAt).toLocaleTimeString([], {
hour: "2-digit",
minute: "2-digit",
});
card.querySelector("pre").textContent = JSON.stringify(message.cryptoContainer, null, 2);
card.addEventListener("click", () => highlightContainer(message.id));
els.containerView.append(card);
}
}

function highlightContainer(messageId) {
const cards = els.containerView.querySelectorAll(".container-card");
cards.forEach((card) => card.classList.toggle("active", card.dataset.messageId === messageId));
const active
els.containerView.querySelector(`.container-card[data-message-id="${CSS.escape(messageId)}"]`);
active?.scrollIntoView({ block: "nearest" });
}

function renderSensors(data) {
els.sensorTopic.textContent = data.topic;
const samples = data.samples.slice(-5).reverse();
els.sensorList.innerHTML = samples
.map(
(sample) => `
<div class="sensor-row">
<strong>${sample.temperature.toFixed(1)} °C</strong>
<span>${sample.humidity.toFixed(1)} %</span>
<span>${sample.pressure.toFixed(1)} hPa</span>
<small>${new Date(sample.created_at).toLocaleTimeString()}</small>
</div>
`,
)
.join("");
}

async function loadUsers() {
const data = await request("/api/users");
users = data.users;
}

```

```

renderUsers();
}

async function loadMessages() {
if (!token || !selectedPeer) return;
const data = await request(`/api/messages?peerId=${encodeURIComponent(selectedPeer.id)}`);
showApp(data.user);
messages = data.messages;
const topic = messages[0]?.chatTopic || `litebroker.chat.${[me.username.slice(1),
selectedPeer.username.slice(1)].sort().join("__")}`;
els.chatTopic.textContent = topic;
renderMessages();
}

async function loadSensors() {
if (!token) return;
const data = await request("/api/sensors");
renderSensors(data);
}

async function health() {
try {
const data = await request("/api/health");
els.serverStatus.textContent = data.kafka.ready ? "Kafka подключена" : "Kafka недоступна, запись локально";
els.serverStatus.classList.toggle("warn", !data.kafka.ready);
} catch {
els.serverStatus.textContent = "сервер недоступен";
els.serverStatus.classList.add("warn");
}
}

els.loginTab.addEventListener("click", () => setMode("login"));
els.registerTab.addEventListener("click", () => setMode("register"));
els.logout.addEventListener("click", logout);
els.selectUserButton.addEventListener("click", () => {
els.userPopover.hidden = !els.userPopover.hidden;
});
els.userSearch.addEventListener("input", renderUsers);

els.authForm.addEventListener("submit", async (event) => {
event.preventDefault();
els.authError.textContent = "";
try {
const data = await request(`/api/auth/${mode === "login" ? "login" : "register"}`, {
method: "POST",
body: JSON.stringify({
displayName: els.displayName.value,
username: els.username.value,
password: els.password.value,

```

```

    }),
  });
  token = data.token;
  localStorage.setItem("litebroker.token", token);
  showApp(data.user);
  await Promise.all([loadUsers(), loadSensors(), health()]);
} catch (error) {
  els.authError.textContent = error.message;
}
});

```

```

async function sendMessage() {
  if (!selectedPeer) return;
  const text = els.messageText.value.trim();
  if (!text) return;
  els.sendButton.disabled = true;
  try {
    const data = await request("/api/messages", {
      method: "POST",
      body: JSON.stringify({ text, recipientId: selectedPeer.id }),
    });
    els.messageText.value = "";
    els.containerView.textContent = JSON.stringify(data.message.cryptoContainer, null, 2);
    await loadMessages();
  } finally {
    els.sendButton.disabled = false;
  }
}

```

```

els.messageForm.addEventListener("submit", async (event) => {
  event.preventDefault();
  await sendMessage();
});

```

```

els.messageText.addEventListener("keydown", async (event) => {
  if (event.key !== "Enter" || event.shiftKey) return;
  event.preventDefault();
  await sendMessage();
});

```

```

setMode("login");
if (token) {
  Promise.all([request("/api/messages").then((data) => showApp(data.user)), loadUsers(), loadSensors(), health()]).catch(logout);
}

```

```

setInterval(() => {
  void health();
  void loadSensors().catch(() => undefined);
}, 1000);

```

```
if (selectedPeer) void loadMessages().catch() => undefined);
}, 3000);
```

LiteBroker\public\styles.css

```
:root {
color-scheme: dark;
--page: #131a22;
--page-soft: #18222d;
--ink: #f4f8fb;
--muted: #aebdca;
--line: #304154;
--line-soft: #425568;
--panel: #18212b;
--panel-2: #202c37;
--panel-3: #101820;
--green: #3d8d47;
--green-dark: #2f6f3d;
--green-soft: #4f9f5a;
--blue: #42a8dc;
--blue-soft: #78c6ee;
--danger: #ff897f;
--shadow: 0 18px 45px rgba(0, 0, 0, 0.28);
font-family: Inter, "Segoe UI", Arial, Helvetica, sans-serif;
}
```

```
* {
box-sizing: border-box;
}
```

```
body {
margin: 0;
min-height: 100vh;
overflow: hidden;
background: var(--page);
color: var(--ink);
}
```

```
[hidden] {
display: none !important;
}
```

```
button,
input,
textarea {
font: inherit;
}
```

```
button {
cursor: pointer;
transition:
background-color 160ms ease,
```



```
border-color 160ms ease,  
color 160ms ease,  
transform 160ms ease,  
box-shadow 160ms ease;  
}
```

```
.shell {  
width: min(1460px, calc(100vw - 32px));  
margin: 16px auto 22px;  
}
```

```
.auth-panel {  
max-width: 520px;  
margin: 7vh auto 0;  
padding: 28px;  
border: 1px solid var(--line);  
border-radius: 22px;  
background: rgba(24, 33, 43, 0.94);  
box-shadow: var(--shadow);  
}
```

```
.auth-panel .tabs button {  
min-height: 50px;  
}
```

```
.auth-panel input {  
min-height: 48px;  
}
```

```
.auth-panel .stack button {  
min-height: 52px;  
}
```

```
.auth-panel .hint {  
margin: 18px 0 0;  
}
```

```
.auth-panel h1 {  
margin: 0 0 8px;  
font-size: 34px;  
}
```

```
.lead,  
.hint {  
color: var(--muted);  
line-height: 1.45;  
}
```

```
.eyebrow {  
margin: 0 0 8px;
```

```
color: #90d7ff;
text-transform: uppercase;
font-weight: 700;
font-size: 12px;
}
```

```
.tabs {
display: grid;
grid-template-columns: 1fr 1fr;
margin: 22px 0;
}
```

```
.tabs button,
.stack button,
.composer button,
.logout-button {
border: 1px solid var(--line);
border-radius: 14px;
background: var(--panel-2);
color: var(--ink);
padding: 13px 14px;
}
```

```
.tabs button.active,
.stack button {
background: var(--blue);
border-color: var(--blue);
font-weight: 700;
box-shadow: 0 10px 24px rgba(66, 168, 220, 0.18);
}
```

```
.stack {
display: grid;
gap: 14px;
}
```

```
label {
color: #c9d7e2;
font-size: 14px;
}
```

```
input,
textarea {
width: 100%;
border: 1px solid var(--line-soft);
border-radius: 14px;
background: #1b2631;
color: var(--ink);
padding: 13px 14px;
margin-top: 7px;
}
```

```
outline: none;
transition:
border-color 160ms ease,
box-shadow 160ms ease,
background-color 160ms ease;
}
```

```
input:focus,
textarea:focus {
border-color: var(--blue-soft);
box-shadow: 0 0 0 3px rgba(66, 168, 220, 0.18);
}
```

```
.error {
min-height: 20px;
color: var(--danger);
}
```

```
.lab-layout {
height: calc(100vh - 38px);
display: grid;
grid-template-rows: auto minmax(0, 1fr);
}
```

```
.lab-topbar {
display: grid;
grid-template-columns: 190px minmax(260px, 1fr) 270px 446px;
align-items: center;
gap: 22px;
min-height: 98px;
margin-bottom: 14px;
}
```

```
.select-wrap {
position: relative;
}
```

```
.select-user-button {
width: 190px;
min-height: 94px;
border: 1px solid var(--line);
border-radius: 20px;
background: rgba(24, 33, 43, 0.92);
color: var(--ink);
text-align: left;
padding: 20px 18px;
font-size: 23px;
line-height: 1.16;
box-shadow: var(--shadow);
}
```

```
}
```

```
.select-user-button:hover {  
border-color: var(--blue-soft);  
background: #1d2935;  
transform: translateY(-1px);  
}
```

```
.user-popover {  
position: absolute;  
z-index: 10;  
left: 0;  
top: 110px;  
width: 320px;  
max-height: calc(100vh - 130px);  
overflow: auto;  
border: 1px solid var(--line);  
border-radius: 20px;  
background: rgba(17, 25, 33, 0.98);  
padding: 14px;  
box-shadow: var(--shadow);  
}
```

```
.peer-name {  
font-size: 25px;  
letter-spacing: 0;  
font-weight: 700;  
line-height: 1.1;  
color: #f3f8fb;  
}
```

```
.session-label {  
margin-top: 8px;  
color: #b5c8d8;  
font-size: 13px;  
line-height: 1.25;  
}
```

```
.crypto-method {  
display: grid;  
justify-items: center;  
align-content: center;  
min-height: 78px;  
text-align: center;  
font-size: 20px;  
color: #e9f4fa;  
}
```

```
.crypto-method strong {  
display: block;  
margin-top: 4px;
```

```
color: var(--blue-soft);
font-size: 14px;
letter-spacing: 0;
}
```

```
.crypto-method small {
display: inline-flex;
align-items: center;
justify-content: center;
max-width: 240px;
min-height: 24px;
margin-top: 8px;
border: 1px solid rgba(157, 228, 168, 0.36);
border-radius: 999px;
background: rgba(61, 141, 71, 0.13);
color: #9de4a8;
padding: 4px 10px;
font-size: 11px;
line-height: 1.15;
text-wrap: balance;
}
```

```
.crypto-method small.warn {
border-color: rgba(255, 137, 127, 0.38);
background: rgba(255, 137, 127, 0.1);
color: #ffa033;
}
```

```
.crypto-heading {
border: 1px solid rgba(255, 255, 255, 0.68);
border-radius: 18px;
background: rgba(244, 248, 251, 0.96);
color: #17202a;
padding: 14px 24px;
font-size: 29px;
box-shadow: 0 14px 32px rgba(0, 0, 0, 0.22);
}
```

```
.chat-stage {
display: grid;
grid-template-columns: minmax(620px, 1fr) 446px;
min-height: 0;
gap: 28px;
}
```

```
.chat-canvas {
display: grid;
grid-template-rows: auto 1fr auto;
min-height: 0;
height: 100%;
}
```

```
border: 1px solid var(--line);
border-radius: 24px;
background:
linear-gradient(180deg, rgba(12, 18, 24, 0.97), rgba(8, 13, 18, 0.98)),
#0c1218;
padding: 24px;
box-shadow: var(--shadow);
}
```

```
.topic-line {
min-height: 22px;
color: #acd9f3;
font-size: 13px;
word-break: break-word;
opacity: 0.9;
}
```

```
.messages {
overflow: auto;
min-height: 0;
padding: 24px 18px;
}
```

```
.message {
width: min(416px, 72%);
min-height: 124px;
margin: 0 18px 34px auto;
border: 1px solid rgba(174, 214, 169, 0.45);
border-radius: 18px 18px 5px 18px;
background: var(--green);
padding: 16px 14px 12px;
color: white;
box-shadow: 0 14px 26px rgba(0, 0, 0, 0.24);
}
```

```
.message p {
margin: 0;
min-height: 64px;
white-space: pre-wrap;
word-break: break-word;
font-size: 23px;
line-height: 1.25;
}
```

```
.message footer {
display: flex;
justify-content: space-between;
gap: 16px;
color: rgba(255, 255, 255, 0.84);
font-size: 13px;
```

```
}
```

```
.message:not(.own) {  
margin-left: 18px;  
margin-right: auto;  
background: var(--green-dark);  
border-radius: 18px 18px 18px 5px;  
}
```

```
.composer {  
display: grid;  
grid-template-columns: 1fr 210px;  
gap: 16px;  
align-items: stretch;  
}
```

```
.composer textarea {  
min-height: 78px;  
margin: 0;  
border: 1px solid var(--line-soft);  
background: #1a2530;  
color: white;  
font-size: 19px;  
resize: none;  
}
```

```
.composer button {  
border: 0;  
border-radius: 18px;  
background: var(--green);  
color: white;  
font-size: 22px;  
font-weight: 700;  
box-shadow: 0 14px 24px rgba(61, 141, 71, 0.22);  
}
```

```
.composer button:not(:disabled):hover {  
background: var(--green-soft);  
transform: translateY(-1px);  
}
```

```
.composer button.disabled {  
opacity: 0.45;  
cursor: not-allowed;  
}
```

```
.crypto-feed {  
max-height: none;  
height: 100%;  
overflow: auto;
```

```
border: 1px solid var(--line);
border-radius: 24px;
background: rgba(9, 14, 20, 0.94);
padding: 16px 18px 20px;
box-shadow: var(--shadow);
}
```

```
.container-card {
margin-bottom: 14px;
border: 1px solid #2d3e50;
border-radius: 16px;
overflow: hidden;
background: #0e151d;
}
```

```
.container-card.active {
border-color: var(--blue-soft);
box-shadow: 0 0 3px rgba(66, 168, 220, 0.14);
}
```

```
.container-card header {
display: flex;
justify-content: space-between;
gap: 14px;
align-items: center;
background: #f3f7fa;
color: #16202a;
padding: 10px 14px;
}
```

```
.container-card strong {
font-size: 21px;
word-break: break-word;
}
```

```
.container-card span {
font-size: 15px;
}
```

```
.container-card pre {
margin: 0;
padding: 15px 12px;
color: #c9e0ff;
white-space: pre-wrap;
word-break: break-word;
font-size: 13px;
line-height: 1.32;
}
```

```
.users {
```



```
display: grid;
gap: 8px;
margin: 12px 0;
}
```

```
.user-item {
display: grid;
grid-template-columns: 44px 1fr;
align-items: center;
gap: 10px;
width: 100%;
border: 1px solid #2b3947;
border-radius: 14px;
background: #16212b;
color: var(--ink);
padding: 9px;
text-align: left;
}
```

```
.user-item.active,
.user-item:hover {
border-color: var(--blue);
background: #1d2b37;
}
```

```
.avatar {
display: grid;
place-items: center;
width: 42px;
height: 42px;
border-radius: 13px;
background: linear-gradient(135deg, #1686bd, #4bb5e5);
font-weight: 700;
}
```

```
.user-item small,
.sensor-card p,
.sensor-row small {
color: var(--muted);
}
```

```
.sensor-card {
border-top: 1px solid #2b3947;
margin-top: 14px;
padding-top: 12px;
}
```

```
.sensor-row {
display: grid;
grid-template-columns: 1fr 1fr;
```

```
gap: 4px;
border: 1px solid #2b3947;
border-radius: 12px;
padding: 8px;
margin-top: 8px;
background: #111a23;
}
```

```
.empty-state {
align-self: center;
justify-self: center;
max-width: 420px;
color: #b9c3cc;
text-align: center;
line-height: 1.5;
}
```

```
.empty-state.inline {
border: 1px solid #2f3d4a;
border-radius: 18px;
background: rgba(24, 33, 43, 0.72);
padding: 22px;
}
```

```
.empty-mini {
color: var(--muted);
padding: 12px;
}
```

```
@media (max-width: 1180px) {
.lab-topbar {
grid-template-columns: 190px 1fr 240px;
}
```

```
.crypto-heading {
grid-column: 1 / -1;
}
```

```
.chat-stage {
grid-template-columns: 1fr;
overflow: auto;
}
```

```
.crypto-feed {
height: 520px;
}
}
```

```
@media (max-width: 760px) {
.shell {
```

```
width: 100%;  
margin: 0;  
}
```

```
.lab-topbar {  
grid-template-columns: 1fr;  
gap: 12px;  
padding: 10px;  
}
```

```
.lab-layout {  
height: 100vh;  
}
```

```
.select-user-button,  
.user-popover {  
width: 100%;  
}
```

```
.chat-stage {  
gap: 12px;  
}
```

```
.chat-canvas {  
min-height: 70vh;  
padding: 12px;  
}
```

```
.message {  
width: 92%;  
margin-bottom: 28px;  
}
```

```
.composer {  
grid-template-columns: 1fr;  
}  
}
```

```
monitoring\prometheus.docker.yml  
global:  
scrape_interval: 5s
```

```
scrape_configs:  
- job_name: litebroker  
metrics_path: /metrics  
static_configs:  
- targets:  
- litebroker:18090  
monitoring\promtail.yml  
server:  
http_listen_port: 9080
```

grpc\_listen\_port: 0

positions:

filename: /tmp/positions.yaml

clients:

- url: http://loki:3100/loki/api/v1/push

scrape\_configs:

- job\_name: litebroker-chat-containers

static\_configs:

- targets: [localhost]

labels:

job: litebroker-chat-containers

service: litebroker

\_\_path\_\_: /var/log/litebroker/chat-containers.jsonl

- job\_name: litebroker-sensor-samples

static\_configs:

- targets: [localhost]

labels:

job: litebroker-sensor-samples

service: litebroker

\_\_path\_\_: /var/log/litebroker/sensor-samples.jsonl